Assignment 1: Rooflines, Arithmetic Intensity and the Memory Hierarchy

Version 1.0

Released Tuesday 20 October 2026 · Due Tuesday 27 October 2026, 11:59 PM IST

Dr. Raj Dandekar and Shubham Panchal

Contents

1 Assignment overview 1

2 Using AI: think first 2

3 What to submit 3

4 Setup 4

5 Numbers and conventions 4

Part A. The roofline on paper (30 points) 6

Part B. The memory hierarchy on paper (15 points) 9

Part C. RoPE through the memory hierarchy (15 points) 11

Part D. Measure it on an H100 (30 points) 13

Part E. Explain the gap (10 points) 16

Bonus (up to 10 points) 16

Before you submit 17

Further reading 17

1 Assignment overview

In Lecture 1 we watched Qwen3-8B on one H100 write 149 tokens a second, when the chip's arithmetic allows about 65,000 (15.2 billion operations a token, as Lecture 1 counted them). The gap came down to one number: how many operations the GPU does for every byte it brings from memory. The H100's bandwidth and its peak arithmetic were fixed when it left the factory. Operations per byte are not. They depend on how the work is organised, which is what batching, precision and the kernel itself change, and that makes arithmetic intensity the number a kernel engineer works on.

This assignment is about that number, and about the memory hierarchy that decides how many bytes a kernel really moves. It asks you to predict before you measure: first on paper, then on

an H100 that you rent from Modal. Then it asks you to explain the difference.

1.1 What you will do

- **Part A, the roofline on paper.** Place operations on the H100's roofline by hand, from a vector add to all of Qwen3-8B, and predict how long each one takes.
- **Part B, the memory hierarchy on paper.** Count the 32-byte sectors a warp's load brings in, the passes a shared-memory read takes, and the bytes a kernel must keep in flight to keep HBM busy.
- **Part C, RoPE through the memory hierarchy.** Follow rotary position embedding from HBM to the registers and back, at prefill and at decode.
- **Part D, measure it on an H100.** Measure your GPU's ceilings, trace the roofline with real matrix multiplications, tune two memory-bound kernels and time four versions of RoPE, all on Modal.
- **Part E, explain the gap.** Take the largest disagreement between your predictions and your measurements and explain it with an experiment of your own.

1.2 What you will not do

You will not write a kernel. Every kernel is provided in `a1.py`, in the same style as Lecture 3's `dog.py`: CUDA C compiled on the GPU machine and launched from Python. You read the code, change the settings marked `TUNE`, run it, and explain what you see. The only code you must write is two short formulas in `plot.py` (Problems 8 and 9), and Problem 11 may ask you to change a few lines of `a1.py`. Writing kernels comes later in the course.

1.3 Time, cost and points

We expect about eight hours of work: about four for the problems on paper, two for the runs, and two for the report and the video. The four experiments together keep an H100 busy for about two minutes. An H100 costs about $3.95 an hour on Modal, billed by the second, so one pass through all four costs about 15 cents, and even ten passes while you tune stay under $2. New Modal accounts get free monthly credits; see modal.com/pricing.

The problems draw on Lecture 1 (the roofline), Lecture 3 (how a GPU runs your code), Lecture 4 (your first CUDA kernels), Lecture 5 (the memory hierarchy) and Lecture 6 (the transpose ladder and measuring kernels).

Part	Problems	Points
A. The roofline on paper	1 to 3	30
B. The memory hierarchy on paper	4 and 5	15
C. RoPE through the memory hierarchy	6	15
D. Measure it on an H100	7 to 10	30
E. Explain the gap	11	10
Total		100
Bonus	B1 and B2	up to 10
Video	Section 3.2	required

2 Using AI: think first

You may use AI, but only after you have thought for yourself first. The thinking is the part of this assignment that cannot be outsourced.

The reason is practical. A kernel engineer is useful because they can predict a number before they measure it and explain it afterwards: in a design review, in an interview, at a whiteboard with no laptop. An answer you did not work out yourself does not build that skill, even when it is right. The rules:

1. Make your own first attempt at every problem before you ask an AI tool about it. Most problems here are a few lines of arithmetic, so an attempt takes minutes, not hours.
2. After that, use AI the way you would use a knowledgeable friend: to check your arithmetic, to explain an error message from Modal, or to explain an idea you have already tried to understand. Do not use it to write your answers, your explanations or your memo for Problem 11.
3. End each problem with one line: *AI use: none*, or what you asked and what you changed because of the answer. An honest line such as "an AI found that I had counted the KV cache once instead of twice" costs you nothing.
4. Every measured number must come from your own runs on your own Modal account.
5. The video (Section 3.2) is where we check all of this. If you cannot explain an answer in your own words, it earns no credit, however correct it is on paper.

3 What to submit

Upload two files to the course portal before the deadline:

- your report, `A1_Firstname_Lastname.pdf`, and
- your video, `A1_Firstname_Lastname.mp4`.

3.1 The report

- At the top: your name, the email you registered with, and the names of anyone you discussed the problems with.
- Answer the problems in order, under their numbers and letters. Show your arithmetic. A number without its working earns little credit, and a wrong number with clear working earns most of it.
- Give every number a unit. Give every figure labelled axes with units, and say which run it comes from.
- End each problem with its AI-use line (Section 2).
- Add an appendix with the summary that each run of `a1.py` printed, its app and run IDs included. The appendix does not count toward the length.
- Typed and handwritten are both fine, as long as a handwritten page is scanned clearly. Aim for 12 to 18 pages before the appendix. We read every report, and a short, clear answer is worth more than a long one.

3.2 The video

Record a video of 8 to 12 minutes in which you go through your solutions the way you worked them out. If you used AI along the way, this is where you show that you understand what it helped with. The video must include:

- your face on camera, at least during the first minute, and your voice throughout, with your screen shared for the report and the terminal;
- a walk through your answers in order, spending the time where you had to think rather than where it was quick;
- one derivation from Problem 3, 5 or 6 redone live on paper or a whiteboard, without reading it from your report;
- your Modal dashboard at modal.com/apps, showing one of the runs in your report: the app whose ID (`ap-...`) that run printed and your appendix lists (never the page with your API tokens);
- the gap you chose for Problem 11 and the experiment that explained it;
- one thing you got wrong at first, and how you found out.

An MP4 at 720p is enough. The video carries no points of its own, but it is required: a submission without one is not graded, and an answer you cannot explain in it earns no credit. We may also invite some of you to a short call to talk through your submission.

3.3 Collaboration, honesty and your Modal key

- You may discuss the lectures and the ideas in this assignment with classmates. The arithmetic, the runs, the report and the video must be your own.
- Do not present another student's numbers, or numbers from this handout, as your measurements. Two runs never print exactly the same numbers, and every results file records its GPU, its time and its run IDs.
- `modal setup` stores your key in `~/.modal.toml`. Never put it in code, your report, a screenshot or your video. If a key leaks, delete it in your Modal settings and run `modal setup` again.

4 Setup

The code is in the folder `assignment1` of the course repository, next to the lecture folders. You need Python 3.10 or newer and a Modal account. You do not need a GPU of your own.

```
pip install modal==1.5.5 matplotlib==3.10.0
modal setup                     # opens your browser once
git clone https://github.com/VizuaraAI/Kernel-Engg
cd Kernel-Engg/assignment1
modal run a1.py::ceilings       # Problem 7
```

If you cloned the repository for a lecture, run `git pull` inside it instead of `git clone`. The first run builds the software image, which takes two or three minutes; later runs reuse it. Each experiment is one command:

Command	Problem	What it measures
`modal run a1.py::ceilings`	7	copy speed against buffer size; the BF16 matmul and FP32 peaks
`modal run a1.py::roofline`	8	$Y = XW$ with W of 4,096 × 12,288, for 1 to 8,192 tokens
`modal run a1.py::memory`	9	strided reads; grayscale with more bytes in flight per thread
`modal run a1.py::rope`	10	RoPE four ways, from 1 token to 131,072
`python plot.py`	7 to 10	draws your figures on your laptop, from `results/*.json`

How `a1.py` measures. Every time comes from CUDA events: a few warm-up launches, then the median of many timed launches. Unless a table says otherwise, the 50 MB L2 is flushed before each timed launch, by writing 256 MB of zeros, so that the data has to come from HBM. The reading for Lecture 6, *How we (try to) benchmark GPU kernels accurately*, explains why each of these steps matters. Your numbers will be close to ours, not identical, and they move a little from run to run.

5 Numbers and conventions

Use the numbers in Tables 1 and 2 throughout. They are the ones from the lectures.

Table 1: The H100 SXM, as in Lectures 1, 3 and 5.

Streaming multiprocessors (SMs)	132
FP32 lanes (CUDA cores)	128 per SM: four quadrants of 32
Tensor cores	4 per SM; each finishes 512 BF16 multiply-adds (1,024 operations) per tick, dense
Clocks	1.98 GHz for the CUDA cores; NVIDIA quotes the tensor-core peak at 1.83 GHz
HBM3	80 GB, 3.35 TB/s
L2 cache	50 MiB (52.4 MB; the lectures say 50 MB), shared by all SMs
Per SM	65,536 32-bit registers (256 KB); 256 KB of SRAM split between the L1 and shared memory; up to 2,048 threads (64 warps) in at most 32 blocks
One load, one warp alone (Lecture 5)	shared memory about 52 ticks; L1 about 62; L2 about 320; HBM about 800
Unit of memory traffic	a 32-byte sector (a "crate" in Lecture 5)
Shared memory	32 banks, each delivering one 4-byte word per tick
An empty kernel	about 2 μs from launch to finish (Lecture 3 measured 1.8 to 2.0 μs)

Table 2: Qwen3-8B.

Hidden size d	4,096
Transformer blocks	36
Attention	32 query heads and 8 key-value heads (grouped-query attention), each of 128 numbers
MLP	SwiGLU, with gate, up and down projections; intermediate size 12,288
Vocabulary	151,936, with separate input and output matrices
Weights	8.19 billion, in BF16 (2 bytes each)
Norms	RMSNorm before attention and before the MLP; a per-head RMSNorm on q and k (QK-norm)
Positions	RoPE with base 1,000,000

Conventions.

- 1 TB/s is 10^{12} bytes a second; 1 GB is 10^9 bytes and 1 MB is 10^6. MiB and GiB, powers of two, appear only as buffer sizes in the code.
- An operation (FLOP) is one multiply or one add, so a multiply-add counts as two. A compare, an exponential or a division counts as one, unless a problem gives other counts.
- Bytes are HBM traffic. Unless a part says otherwise, each input is read from HBM once and each output is written to HBM once: the minimum.
- Arithmetic intensity is operations divided by bytes, in FLOP per byte.
- A roofline prediction is the longer of two times: operations divided by the peak, and bytes divided by the bandwidth. Use the datasheet numbers in Parts A to C.
- A tick is one clock cycle of an SM.

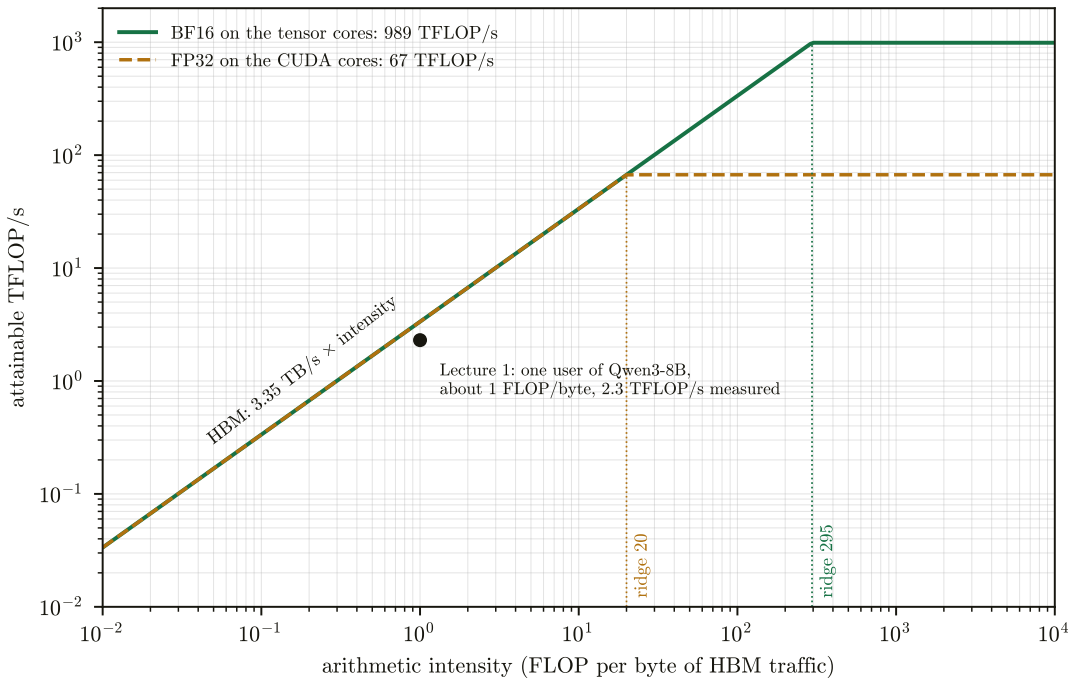


Figure 1: The H100 roofline from Table 1, on log-log axes. Arithmetic on the tensor cores lives under the green roof and arithmetic on the CUDA cores under the dashed one; both share the slanted memory roof. Print it and draw on it, or make your own.

Part A. The roofline on paper (30 points)

Lecture 1 ended on a recipe: count the operations, count the bytes, divide, compare with the ridge, and predict the time as the longer of the memory time and the compute time. Then measure, and explain the gap. Part A applies the recipe on paper. Part D measures.

Problem 1 (`ridge`): Two ceilings, two ridges **6 points**

(a) **(2 points)** Using Table 1, compute the H100's peak for dense BF16 on the tensor cores and for FP32 on the CUDA cores, each as a product of its parts. Then compute the two ridge points against HBM.

Deliverable: two peaks and two ridges, each with its product written out.

(b) **(2 points)** An A100 (80 GB, SXM) has 108 SMs. Each SM has 64 FP32 lanes and 4 tensor cores, and each tensor core finishes 256 BF16 multiply-adds per tick. Both run at 1.41 GHz, and its HBM2e delivers 2.04 TB/s. Compute its two peaks and its two ridges.

Deliverable: the same four numbers for the A100.

(c) **(2 points)** From the A100 to the H100, by what factor did the BF16 peak grow, and by what factor the bandwidth? A kernel whose arithmetic intensity equals the A100's BF16 ridge runs on the H100: what fraction of the H100's BF16 peak can it reach at best? In two sentences, what does this trend mean for the work of a kernel engineer?

Deliverable: two factors, one fraction and two sentences.

Problem 2 (`intensity`): Arithmetic intensity of six operations **10 points**

	Operation	Type	Size
1	Vector add, $c = a + b$ (Lecture 4's `vecadd`)	FP32	$n = 2^{28}$ elements
2	ReLU, $y = \max(x, 0)$	BF16	$n = 2^{28}$ elements
3	Softmax of every row	FP32	an 8,192 × 8,192 matrix
4	GEMV, $y = xW$ (Qwen3-8B's up projection, one token)	BF16	x: 1 × 4,096, W: 4,096 × 12,288
5	GEMM, $C = AB$	BF16	all three 4,096 × 4,096
6	Transpose	FP32	an 8,192 × 8,192 matrix

Count ReLU as one operation per element, and softmax as five per element (maximum, subtract, exponential, add, divide), with each row read once and written once.

(a) **(6 points)** For each operation find the operations, the bytes, the arithmetic intensity, which roof applies (tensor cores or CUDA cores), whether it is memory-bound or compute-bound, and the predicted time on the H100. For the transpose, say what you can and cannot compute.

Deliverable: one table, with a row for each operation.

(b) **(2 points)** Put the five operations that do arithmetic on one roofline, each under the roof that applies. Where would the transpose go, and how would you judge whether a transpose kernel is a good one?

Deliverable: a roofline with five labelled dots, and two sentences.

(c) **(2 points)** Same arithmetic, different bytes. (i) A softmax kernel reads each row from HBM three times (once for the maximum, once for the sum of exponentials, once to normalise) and writes it once. What are its intensity and its predicted time? What lets a kernel read each row only once, and why does a row of 8,192 FP32 numbers make that easy? (ii) A GEMM kernel fetches both operands of every multiply-add from HBM, with no reuse at all. What are its bytes, its intensity and its predicted time, and how many times slower is it than your answer in (a)? In one sentence, what does a good GEMM kernel do that this one does not?

Deliverable: two short answers with their arithmetic.

Problem 3 (`napkin`): The napkin audit of Qwen3-8B — 14 points

Lecture 1 left you a napkin audit of a 7B model. Here it is in full, for Qwen3-8B (Table 2), one transformer block at a time. Split a block's work into four families:

(i) the attention projections: Q, K, V and O;
(ii) attention itself: the scores $QK^\top$, the softmax and the weighted sum PV, against the KV cache;
(iii) the MLP: the gate, up and down projections;
(iv) the elementwise operations and norms: two RMSNorms, QK-norm on q and k, RoPE on q and k, SiLU($gate$) $\times$ up, and two residual adds.

Treat Q, K and V as one matmul with a 4,096 × 6,144 weight, and gate and up as one with a 4,096 × 24,576 weight, as serving engines do; O and down are matmuls of their own. Each matmul reads its input and its weights once and writes its output once. In family (ii), count 5 operations per score for the softmax, and use 4,096 keys at decode. For family (iv), use these operations per output element: RMSNorm 4, QK-norm 4, RoPE 3, SiLU($gate$) $\times$ up 5, residual add 1. Treat each of these as its own kernel that reads its inputs from HBM and writes its output to HBM, and leave out the norms' small weight vectors and RoPE's tables. Compare family (iv) with the CUDA-core roof and the other three with the tensor-core roof. There are two phases:

- **Decode:** one new token for one user, with 4,096 earlier tokens in the KV cache.
- **Prefill:** a prompt of 2,048 tokens for one user. Attention is causal, so count $S^2/2$ query-key pairs, and it is computed the way FlashAttention does it: the $S \times S$ scores never reach HBM.

(a) **(6 points)** For each family in each phase, find the operations, the bytes, the arithmetic intensity, the bound and the predicted time for one block. Put the eight dots on one roofline.

Deliverable: two tables (decode and prefill) and one roofline with eight labelled dots.

(b) **(3 points)** Decode, the whole model. Add up one token's predicted time through all 36 blocks and the output layer (4,096 × 151,936 weights, read in full for every token). What bound on tokens per second does that give? Redo it with 128 tokens of context, close to Lecture 1's measurement, and compare with the 149 tokens a second measured there. What fraction of your bound did vLLM reach? Name the three causes that you think cost the most of the rest.

Deliverable: two bounds, one fraction, and three causes in a sentence each.

(c) **(2 points)** Prefill. What share of a block's predicted time goes to family (iv), and what share of its operations? What does the difference tell you about where kernel work pays off?

Deliverable: two percentages and two sentences.

(d) **(3 points)** Raising operations per byte. Lecture 1 named three levers: more users in a batch, lower precision, and the kernel itself. Now B users decode together, each with 4,096 tokens of context. (i) Write the arithmetic intensity of the up projection ($B \times 4{,}096$ times $4{,}096 \times 12{,}288$) as a function of B, and find the smallest B that reaches the BF16 ridge. (ii) What happens to the intensity of family (ii) as B grows, and why? At what B does reading the users' KV caches take longer than reading the block's weights? (iii) Redo (i) with the weights stored in 8 bits, and then in 4 bits, while the activations and the arithmetic stay in BF16.

Deliverable: three short answers with their arithmetic.

Part B. The memory hierarchy on paper (15 points)

Lecture 5 followed one pixel of the dog from HBM to a lane and back. Two of its facts run through Part B. Memory answers in 32-byte sectors, never in single bytes. And a load takes hundreds of ticks to come back, so a kernel keeps the memory busy only if it has enough loads on their way at once. Figure 2 puts the levels side by side.

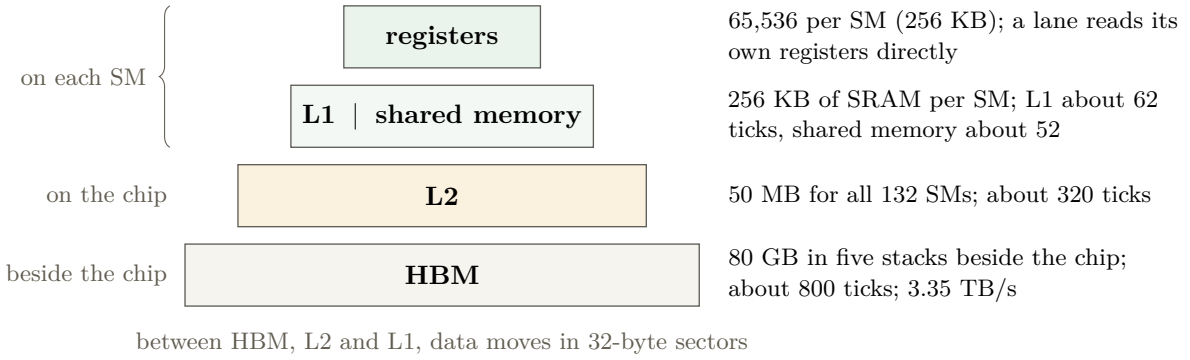


Figure 2: The H100's memory hierarchy, with the latencies Lecture 5 measured for one warp alone. The higher a level, the smaller and faster it is. The L1 and the L2 keep recently used sectors by themselves; shared memory holds only what a program puts there.

Problem 4 (`sectors`): Sectors, coalescing and banks — 8 points

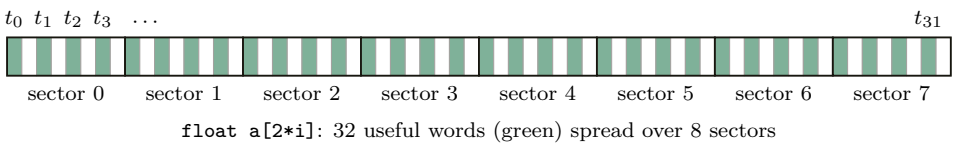


Figure 3: One warp reading every other float. Each box is a 4-byte word; memory sends whole 32-byte sectors.

(a) **(4 points)** For each warp-wide load below, how many 32-byte sectors does the warp touch, how many bytes does memory move, and what fraction of them does the program use? Thread i is lane i of the warp, and every array starts on a 128-byte boundary.

1. `float a[i]`
2. `float a[2*i]` (Figure 3)
3. `float a[32*i]`
4. `a[i]`, with `a` in BF16
5. `float a[i+1]`
6. `uchar4` pixels of the 1,024-wide dog, read by one warp of Lecture 3's 16×16 blocks: lanes 0 to 15 read 16 neighbouring pixels of one row, and lanes 16 to 31 the 16 pixels below them. The block's first column is a multiple of 16.
7. `float A[i][c]`, with `A` row-major and $8{,}192 \times 8{,}192$: one column
8. `uint4 p[i]`: 16 bytes for each thread

Deliverable: a table with three numbers for each of the eight loads.

(b) **(4 points)** Shared memory has 32 banks, and the 4-byte word at index w lives in bank w mod 32. A warp's read takes as many passes (ticks at the banks) as the largest number of different words it needs from one bank. How many passes does each read take?

1. `float tile[32][32]`; thread i reads `tile[i][c]`.
2. `float tile[32][33]`; thread i reads `tile[i][c]`.
3. `float tile[32][34]`; thread i reads `tile[i][c]`.
4. `float tile[32][32]`; thread i reads `tile[c][i]`.

Lecture 6's transpose removes its bank conflicts by padding its tile. Explain why one extra column is enough, and why two extra columns would be worse.

Deliverable: four numbers and two sentences.

Problem 5 (`inflight`): Bytes in flight — 7 points

Lecture 5 ended with a challenge. Its grayscale kernel reads one 4-byte pixel per thread and keeps 2,048 threads on every SM, yet on 64 dogs (50,331,648 pixels) it took 164.7 μs: 1.53 TB/s, counting 4 bytes read and 1 byte written per pixel, under half of 3.35 TB/s. A version that reads 16 bytes (4 pixels) per thread took 84.2 μs: 2.99 TB/s. Little's law explains the difference. To sustain a bandwidth W when each request takes a time T to come back, $W \times T$ bytes must be on their way at every moment.

(a) **(2 points)** Take $T = 800$ ticks at 1.98 GHz. How many bytes must be in flight across the whole H100 to reach 3.35 TB/s, and how many on each SM?

Deliverable: two numbers with their arithmetic.

(b) **(2 points)** At most how many bytes can the one-pixel kernel have in flight on an SM, and what read bandwidth does Little's law then allow with $T = 800$ ticks? Its measured read bandwidth (4 of every 5 bytes) is lower. What latency T does Little's law imply from the measurement? Give two reasons why it is longer than 800 ticks.

Deliverable: three numbers and two reasons.

(c) **(2 points)** Only loads are in flight in this sense: a thread does not wait for its stores.

With the latency you found in (b), how many bytes of loads must each of the 2,048 threads on an SM have in flight for the kernel to reach 90% of 3.35 TB/s, counting its reads and writes in that 90% as the lecture did? Does the 16-byte kernel's result agree?

Deliverable: one number and one sentence.

(d) **(1 point)** Name two ways, other than wider loads, to put more bytes in flight. Problem 9 measures one of them.

Deliverable: two ways, in a sentence each.

Part C. RoPE through the memory hierarchy (15 points)

Qwen3-8B tells each query and key vector where its token sits by rotating it. Split a head vector x of 128 numbers into two halves. For each pair (x_i, x_{i+64}) with $i < 64$, at position p, with angle $\theta_{p,i} = p \cdot 1{,}000{,}000^{-2i/128}$:

$$x'_i = x_i \cos\theta_{p,i} - x_{i+64} \sin\theta_{p,i}, \qquad x'_{i+64} = x_{i+64} \cos\theta_{p,i} + x_i \sin\theta_{p,i}.$$

The cosines and sines are computed once into tables indexed by position and pair. RoPE runs on q (32 heads) and k (8 heads) in each of the 36 blocks, for every token. It has almost no arithmetic, which is why it is a good way to see the memory hierarchy at work: nearly all of its cost is in moving bytes and launching kernels.

Setup. Prefill of $S = 4{,}096$ tokens. q has shape $[S, 32, 128]$ and k has shape $[S, 8, 128]$, both in BF16 and stored token by token: all heads of token 0, then all heads of token 1, and so on. The fused kernel reads its angles from two tables, `cos` and `sin`, each of shape $[S, 64]$ in FP32.

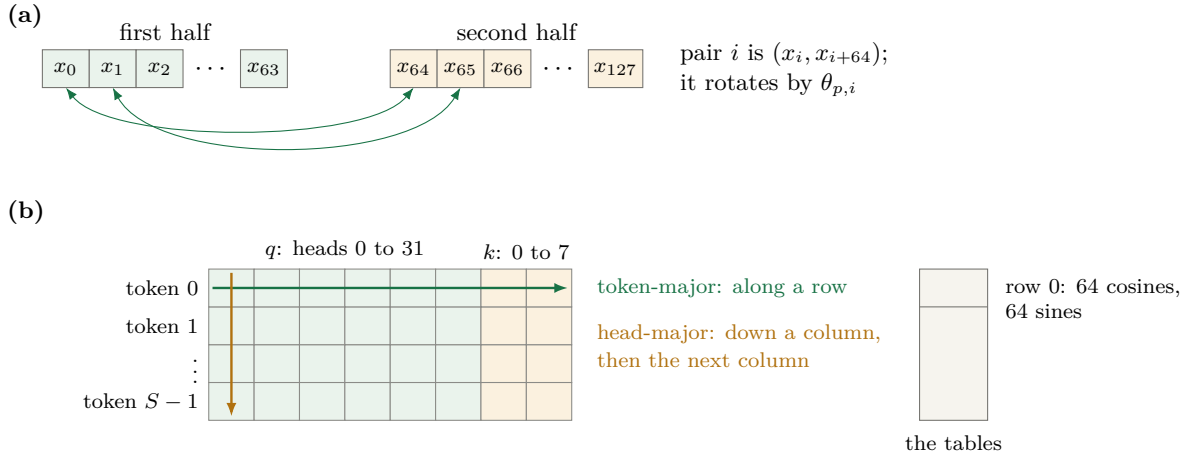


Figure 4: (a) RoPE rotates pair (x_i, x_{i+64}) of a head vector by an angle that depends on the token's position p and on i. (b) q and k as a grid of head vectors of 256 bytes each, one row per token. Every head vector in row p uses row p of the tables. A kernel can walk the grid along the rows (token-major) or down the columns (head-major).

Problem 6 (`rope`): RoPE on paper — 15 points

(a) **(3 points)** A fused kernel reads q and k once, rotates each pair in registers and writes the results back in place. Count its operations, its bytes (q and k each read once and

written once, and both tables read once), its arithmetic intensity and the roof that applies. Predict its time.

Deliverable: operations, bytes, intensity, the roof, and a predicted time.

(b) **(3 points)** Hugging Face's Llama and Qwen3 code writes RoPE as

```
q * cos + rotate_half(q) * sin        # where
rotate_half(x) = cat(-x[..., 64:], x[..., :64])
```

with `cos` and `sin` of shape $[S, 1, 128]$ in BF16. In eager PyTorch every operator is its own kernel, which reads its inputs from HBM and writes its result to HBM. List the kernels this launches for q and the bytes each one moves (the tables come from L2, so leave them out). Do the same for k. Predict the total time. How many times more bytes of q and k traffic does it move than the fused kernel of (a), leaving the tables out of both?

Deliverable: the kernels with their bytes, a total, a predicted time and the factor.

(c) **(4 points)** Where the tables live. (i) How large are the two tables together at $S = 4{,}096$, and at $S = 131{,}072$? (ii) The fused kernel can hand out its work in two orders (Figure 4). Token-major: neighbouring threads take the 40 heads of one token, then move to the next token. Head-major: they take one head through all S tokens, then move to the next head. In each order, how many bytes does the kernel read or write (q, k and table rows together) between two uses of the same row of the tables? (iii) Which level of the hierarchy serves the second use, in each order, at $S = 4{,}096$ and at $S = 131{,}072$? (iv) Predict the HBM traffic and the time of both orders at $S = 131{,}072$. Which order would you ship?

Deliverable: four short answers with their arithmetic.

(d) **(3 points)** Decode. Now $S = 1$: a single new token. (i) How many bytes does the fused kernel move, and what time does the roofline predict? What did Lecture 3's empty kernel take, and what really bounds this one? (ii) Every generated token of Qwen3-8B runs RoPE in all 36 blocks, and eager PyTorch launches 10 kernels for it in each block. At about 2 μs a launch, how long is that per token, and what share of the 6.7 ms per token that Lecture 1 measured (149 tokens a second)? (iii) Name two ways a serving engine removes most of this cost.

Deliverable: three short answers.

(e) **(2 points)** The cheapest RoPE. (i) If the kernel of the QKV projection applied RoPE to q and k just before writing them out, how many extra HBM bytes and extra launches would RoPE cost? (ii) Instead of reading the tables, a kernel can compute each cosine and sine from p and i. Suppose that costs 20 operations per pair. What is the new arithmetic intensity, and does RoPE become compute-bound? Say which roof you compared against, and why that one.

Deliverable: two short answers.

Part D. Measure it on an H100 (30 points)

Each problem below is one command (Section 4), and each asks you to set what you measure against what you predicted in Parts A to C. Use the medians that `a1.py` prints and say which run they come from. When you change a `TUNE` setting, say what you changed.

Problem 7 (`ceilings`): Your H100's ceilings, and its L2 **6 points**

`modal run a1.py::ceilings` times `dst.copy_(src)` for buffers from 1 MiB to 4 GiB in two ways. *Warm*: many copies back to back, so the L2 keeps both buffers if they fit. *Cold*: the L2 is flushed before each copy. It also times square BF16 matrix multiplications from 1,024 to 16,384, and a kernel that does nothing but FP32 multiply-adds.

(a) **(3 points)** Plot copy speed against buffer size (`plot.py` draws it). (i) For which buffer sizes does the warm copy beat HBM's 3.35 TB/s, and how does that range relate to the 50 MB L2? (ii) Why do the smallest buffers fall far below 3 TB/s, warm or cold? (iii) What fraction of 3.35 TB/s do the largest buffers reach?

Deliverable: the figure and three short answers.

(b) **(2 points)** Report your measured ceilings: the HBM bandwidth (the cold copy of the largest buffer), the best BF16 matmul and the FP32 kernel, each as a fraction of the datasheet, and your two measured ridges. Why is the 1,024 × 1,024 matmul so far below the larger ones? Count its output tiles if each SM works on 128 by 128 outputs at a time, and compare with the 132 SMs.

Deliverable: a small table and two sentences.

(c) **(1 point)** Run `ceilings` a second time. Which numbers moved, and by how much? To how many significant figures would you trust each one?

Deliverable: two sentences.

Problem 8 (`batch`): Trace the roofline and choose a batch size **8 points**

`modal run a1.py::roofline` times $Y = XW$ in BF16, where W is Qwen3-8B's up projection (4,096 × 12,288) and X holds M tokens (M × 4,096), for $M = 1, 2, 4, \ldots, 8{,}192$, and for a finer sweep set by `FINE_TOKENS`.

(a) **(3 points)** Write the arithmetic intensity of this matmul as a function of M, and complete `arithmetic_intensity()` in `plot.py` with it. Place your points on your measured roofline (`plot.py` draws it once the function returns a number). Which points sit under the slanted roof, which near the flat one, and where does the turn happen?

Deliverable: the formula, the figure and two sentences.

(b) **(2 points)** Look at the times from $M = 1$ to $M = 256$. What does it cost to put 256 tokens through this layer instead of 1? Explain it with the roofline, and say what it means for a server with many users.

Deliverable: one ratio and three sentences.

(c) **(2 points)** With the fine sweep (change `FINE_TOKENS` if you like), find M^*, the smallest

batch size whose time is at least 10% above the time at $M = 1$. Compare it with the turn your measured roofline predicts and with Problem 3(d)(i). Is the time always larger for a larger batch? If you find a batch size that runs slower than a larger one, give a reason this can happen and say what a serving engine should do about it.

Deliverable: M^*, one prediction, and three or four sentences.

(d) **(1 point)** At $M = 1$, what fraction of the H100's BF16 datasheet peak does the matmul reach, and what fraction of your measured copy bandwidth? Relate the first to Lecture 1's 0.23%.

Deliverable: two fractions and one sentence.

Problem 9 (`coalesce`): Coalescing and bytes in flight, measured **8 points**

`modal run a1.py::memory` runs two experiments.

(a) **(4 points)** *Strided reads.* The kernel copies 2^{24} floats: thread i reads `src[i*stride]` and writes `dst[i]`. (i) From your sector counts in Problem 4, predict the useful bandwidth for each stride (8 bytes per float: the read and the write), assuming memory moves bytes at the speed your stride-1 run reached. (ii) Plot your prediction against the measurement (`plot.py` draws it once you complete `strided_prediction()`). (iii) For which strides does the measurement beat your prediction, and for which does it fall below it? Give a cause for each; Problem 5 and Lecture 5's film on memory rows will help.

Deliverable: the figure, and a cause for each side of the prediction.

(b) **(4 points)** *Bytes in flight.* The run times Lecture 5's grayscale on 64 dogs with `VEC` pixels per load and `ILP` independent loads per thread, for four block sizes. (i) Which configuration is fastest, and how close does it come to your copy bandwidth from Problem 7? (ii) Plot TB/s against the bytes each thread loads at once ($4 \times$ `VEC` $\times$ `ILP`) and explain the shape with Problem 5. Does it matter whether the bytes come from wider loads or from more loads? (iii) From your `VEC` $= 1$, `ILP` $= 1$ result with 256 threads a block, what loaded latency does Little's law imply on your GPU, in nanoseconds and in ticks? (iv) With 4 bytes a thread the block size changes the speed, and with 16 or more it hardly does. Why?

Deliverable: the figure and four short answers.

Problem 10 (`rope_measured`): RoPE, measured **8 points**

`modal run a1.py::rope` applies RoPE to Qwen3-8B's q and k for S from 1 to 131,072 tokens in four ways: a plain copy of the same bytes (a speed limit, not RoPE), eager PyTorch as Hugging Face writes it, `torch.compile` of the same function, and a fused CUDA kernel. It also times the CUDA kernel in both orders of Problem 6(c) with 1 to 8 pairs per thread, and calls each version back to back at small S to show what a program pays per call.

(a) **(3 points)** At $S = 4{,}096$, compute each version's achieved bandwidth twice: with the minimum bytes from Problem 6(a), and with the bytes that version really moves (Problem 6(b) for eager; take the minimum for `torch.compile` and the CUDA kernel). Compare each with the copy. Where did your Problem 6 predictions hold, and where not?

Deliverable: a table and three sentences.

(b) **(2 points)** Plot time against S for the four versions (`plot.py`). Below which S does the time stop falling, and what sets that floor? Now read the back-to-back table at $S = 1$, 16 and 128: what does a program pay per call for each version, and how much of that is the GPU's own work? Compare with Problem 6(d).

Deliverable: the figure and three sentences.

(c) **(2 points)** The order test. Explain the token-major and head-major times at $S = 4{,}096$ and $S = 131{,}072$ with your answer to Problem 6(c). Which number of pairs per thread is fastest, and why (Problem 5)?

Deliverable: two short explanations.

(d) **(1 point)** A teammate says: "RoPE is less than 0.01% of Qwen3-8B's operations, so it cannot matter." Reply in at most three sentences, using your measurements: when are they right, and when are they wrong?

Deliverable: three sentences.

Part E. Explain the gap (10 points)

Problem 11 (`gap`): One gap, one experiment **10 points**

Pick the largest disagreement between a prediction you made in Parts A to C and a measurement you took in Part D, judged by the ratio, not the difference. Write it up in at most one page:

1. the prediction, with its arithmetic;
2. the measurement, and how it was taken;
3. at least two hypotheses that could explain the gap;
4. one experiment of your own that tells them apart: change a `TUNE` setting, a size or a few lines of `a1.py`, run it, and report the result;
5. your conclusion, and how sure you are of it.

We grade the reasoning, not the size of the gap: 2 points for stating the prediction and the measurement clearly, 2 for specific hypotheses that an experiment could tell apart, 4 for the experiment (does it separate the hypotheses, is it measured properly, is its result reported as it came out), and 2 for a conclusion that claims no more than the evidence shows. A careful test that rules out your favourite hypothesis earns more than a confident story with no test.

Deliverable: one page, with the extra run's printout in your appendix.

Bonus (up to 10 points)

Bonus B1 (`second_gpu`): A second GPU **up to 5 points**

Set `GPU = "L4"` (or `"A100-80GB"`) near the top of `a1.py` and run `ceilings` and `roofline` again; `a1.py` saves those runs next to your H100 results (for example `results/ceilings_L4.json`), so nothing is overwritten. An L4 costs far less an hour than an H100; check Modal's pricing page first. Draw both measured rooflines on one figure yourself (`fig_roofline` in `plot.py` is a starting point). Where did the ridge move, and what does that mean for which kernels are memory-bound on each GPU? Use the GPU's own datasheet numbers, and say where you found them.

Deliverable: one figure and a short paragraph.

Bonus B2 (`profile`): Check one prediction with a profiler **up to 5 points**

Check one prediction from Parts B or C against a profiler's count. For example, count the kernels that eager and compiled RoPE launch with `torch.profiler` and compare with Problem 6(b); or, if you can run Nsight Compute as in Lecture 6, compare the sectors per request of one stride in Problem 9(a) with your Problem 4 count. Show the command, the counter's value and your prediction.

Deliverable: the command, the profiler's output and two sentences.

Before you submit

- Every problem is answered under its number, with its arithmetic and an AI-use line.
- Every figure has labelled axes with units and names the run it comes from.
- The appendix holds the printed summary of every run you used, with run IDs.
- Your video is 8 to 12 minutes long and has the six things listed in Section 3.2.
- Nothing in your report or video shows a Modal key.
- The two files are named `A1_Firstname_Lastname.pdf` and `A1_Firstname_Lastname.mp4`.

Further reading

S. Williams, A. Waterman and D. Patterson, *Roofline: an insightful visual performance model for multicore architectures*, Communications of the ACM, 2009.
H. He, *Making Deep Learning Go Brrrr From First Principles*, 2022.
J. Su et al., *RoFormer: Enhanced Transformer with Rotary Position Embedding*, 2021.
Modal, *GPU Glossary*, modal.com/gpu-glossary.
Jan.ai, *How we (try to) benchmark GPU kernels accurately*, the reading for Lecture 6.
The lecture decks and the book chapters for Lectures 1, 3 and 5, in the course on Vizuara.